

开始深度研究前的范围确认

我已经先做了一轮只用于消除歧义的预检索，目前确认你提到的核心工作都是真实且高度相关的：OPT-Engine 明确把 benchmark 设计成可扩展、可控制复杂度的 optimization modeling 测试框架，并报告 solver-integrated reasoning 随规模增长比纯文本求解稳健得多，同时将 **constraint formulation** 定位为主要瓶颈；ORAgentBench 则在 2026 年提出了包含 107 个真实端到端 OR 任务的 execution-grounded benchmark；LEAN-LLM-OPT 聚焦大规模优化模型自动 formulation，并引入 Large-Scale-OR 与 Air-NRM。除此之外，近两年还出现了 OR-LLM-Bench、OR-LLM-Agent、OR-R1、OptiBench、CO-Bench、Opti-Agent-Bench 等值得纳入综述的工作。¹

在真正开始你要求的长篇文献综述、数学定义和实验设计之前，我还剩下一组会实质改变论文定义的歧义。其中最关键的不是 benchmark 名单，而是“problem-intrinsic / model-independent”与“从 formalized model 计算 complexity”之间存在一个必须先解决的逻辑张力。

我目前已经确定的理解

基于你之前的 Q1-Q12，我暂时把你的研究目标理解为：

对一个由 **natural-language problem description + structured/unstructured data + business rules** 构成的完整 OR instance I ，定义一个 **problem-intrinsic、尽量 model-independent** 的多维复杂度向量

$$> C_{\text{op}}(I) >=> (> C_{\text{sem}}, > C_{\text{couple}}, > C_{\text{feas}}, > C_{\text{amb}}, > \dots >), >$$

它不以某一个具体 LLM 的行为作为定义本身，而应该能够**预测不同 LLM / agent / solver-integrated system 在该 instance 上的 performance degradation**。

LLM accuracy、feasibility rate、formalization accuracy 等是这个 complexity measure 的**外部效度验证变量**，而不是 complexity 的定义。

同时，你目前最重要的科学假设是：

对于 solver-integrated OR solving，主要困难可能集中在 problem semantics $\rightarrow$ correct formalization

而不是

$$\text{correct formalization} \rightarrow \text{numerical/combinatorial solution.}$$

你特别希望检验更强的版本：

$$P(\text{correct final answer} \mid \text{gold formalization}) \gg P(\text{correct final answer} \mid \text{natural-language instance}),$$

并进一步判断这种差异是否足够大、足够普遍，以支持将 **operational problem complexity** 的主体定义为 **semantic/modeling complexity**，而将经典 computational complexity 单独处理。

OPT-Engine 的结论确实朝这个方向提供了初步证据，但它目前覆盖的十类任务仍主要处在 LP/MIP canonical problems 范围，因此不能直接推出该命题对 routing、scheduling、CP、stochastic optimization 或大型

combinatorial instances 普遍成立。OPT-Engine 自己的核心发现是：solver integration 对规模扩展十分重要，而约束自动 formulation 是 SIR 的主要瓶颈；这更接近“当前 benchmark 范围内的经验命题”，还不是关于 OR problem complexity 的一般性定理。²

必须确认的核心问题：什么叫 model-independent

这是现在最重要的一点。

同一个 operational problem 可以有多个完全等价、但结构差异极大的 formulation。

例如一个 scheduling problem 可以写成：

$$x_{jt} \in \{0, 1\},$$

做 time-indexed MILP；

也可以写成 pairwise precedence variables；

还可以使用 interval variables + CP-SAT；

甚至可以使用 column-generation / set-partitioning formulation。

这些表示的：

- constraint 数量；
- variable 数量；
- incidence graph；
- treewidth；
- spectral radius；
- overlap；
- density；

都可能大不相同。

因此，如果我们直接从某一个 gold MILP 的 constraint matrix A 计算 coupling：

$$C_{\text{couple}}(A),$$

严格来说这并不是 **problem-intrinsic**，而是：

formulation-dependent complexity.

我建议论文明确区分两层：

Operational Problem Complexity

与

Optimization Formulation Complexity.

第一层定义在原始 operational semantics 上；第二层定义在某一个数学 formulation 上。

然后研究：

$$C_{\text{form}}(M) \approx f(C_{\text{op}}(I))$$

以及

$$C_{\text{op}}(I) \rightarrow P(\text{LLM generates correct } M).$$

请确认你是否接受这种“双层定义”。

我的推荐是：接受。

否则“model-independent”与“从 constraint matrix / hypergraph 计算指标”在理论上很难同时成立。

canonical semantic entities 到底由谁定义

你之前选择了 semantic coupling，并希望数学表达，但也担心 natural-language rules 的语义耦合是否能够稳定量化。

我目前准备采用的核心结构是 **Semantic Obligation Hypergraph**：

$$H_I = (V, E).$$

这里顶点不是 MILP variables，而是 operational instance 中的 **canonical semantic decision entities / state entities / resource entities**。

例如 vehicle-routing-with-time-windows 中可能存在：

$V = \{\text{vehicle assignment, customer visitation, route order, vehicle capacity, arrival time, service time, depot bal}$

一句 business rule：

Every customer must be served exactly once by one vehicle, and a vehicle may not exceed capacity.

会产生多个 semantic obligations，并形成 hyperedges。

这样 coupling 是从**业务语义依赖关系**而不是具体 MILP encoding 定义的。

但这会产生一个问题：

谁来决定两个描述是否属于同一个 canonical entity？

为了保持可验证性，我建议研究第一阶段不要强求完全自动 extraction，而采用：

expert-defined ontology + deterministic annotation guideline + LLM-assisted extraction.

也就是说，论文的 gold complexity score 来自规范化 annotation，而自动计算只是后续任务。

请确认这一点是否接受。

我的推荐仍然是：接受人工/专家 gold semantic graph，自动抽取作为第二层实验。

这会让论文更严谨，否则 semantic complexity 很容易退化“用一个 LLM 来评价另一个 LLM 的难度”。

constraint coupling 是否需要同时保留“语义耦合”和“数学耦合”

我准备把 constraint coupling 拆成两种互补概念：

Semantic coupling

从原始 business rules / semantic obligations 得到：

$$H_{\text{sem}} = (V_{\text{sem}}, E_{\text{sem}}).$$

它回答：

一个业务规则需要同时协调多少种决策实体？不同规则之间共享多少实体？局部依赖如何组成全局依赖？

这是你的 **problem-intrinsic complexity** 主体。

Formulation coupling

从 gold formulation 的 variable-constraint incidence structure 得到：

$$H_{\text{form}} = (V_c, E_c),$$

其中变量是 mathematical variables，constraint 是 hyperedges。

它回答：

在一个给定数学模型中，约束之间通过变量产生了多强的交互？

这里可以计算：

mean overlap, $\rho(\tilde{A})$, treewidth, hubness, cycle density.

然后我们可以研究：

$$C_{\text{sem}} \rightarrow C_{\text{form}} \rightarrow \text{LLM formulation error}.$$

这甚至可以形成一个很有意思的 mediation analysis。

请确认：是否采用这两个 coupling 层次，而不是试图用单一 coupling 指标同时承担二者。

我的推荐是：采用双层 coupling。

“gold formalized model” 实验到底把什么交给 LLM

这是你整篇论文最关键的因果实验之一，需要精确定义。

我建议不是只做两组，而是做一个**逐层 intervention ladder**：

NL instance $\rightarrow$ semantic specification $\rightarrow$ math formulation $\rightarrow$ solver code $\rightarrow$ solution.

于是设置至少四个条件。

条件	提供给模型的输入	LLM仍需完成
A	Natural language + data + rules	semantic parsing + formulation + coding + solving
B	Canonical semantic specification	formulation + coding + solving
C	Gold mathematical model	solver coding + solving
D	Gold executable solver model/code	仅运行/读取/提交答案

最重要的是比较：

$$A \rightarrow C.$$

如果：

$$Acc_C \approx 1$$

而

$$Acc_A \ll Acc_C,$$

则强烈支持 modeling bottleneck hypothesis。

但如果：

$$Acc_C$$

在 scheduling / routing / nonlinear / stochastic / decomposition-heavy problems 上仍显著下降，那么就说明：

$$C_{\text{op}} = C_{\text{modeling}} + C_{\text{solving}}$$

至少需要保留第二部分。

请确认你是否接受这一四级 intervention，而不只做“NL vs gold model”两组。

我的推荐是：**接受四级设计**。它会比简单两组实验强很多，因为能够定位 performance loss 出现在 pipeline 的哪一个阶段。

在 gold-model 条件下，solver 是否固定

这也是一个容易污染实验结论的变量。

假设 gold model 是 MILP，但让 agent 自己决定：

- Gurobi;

- CPLEX;
- SCIP;
- OR-Tools;
- CP-SAT;
- 自己写 heuristic;

那么 solver-selection ability 会混入结果。

为了检验你真正关心的命题：

“只要 formalization 正确，后面是否基本是机械的？”

我建议第一阶段严格固定：

modeling language + solver + time limit + solver parameters.

例如：

Pyomo + Gurobi/SCIP

或 CP 类：

OR-Tools CP-SAT.

然后另开一个实验允许 agent 自己进行 solver selection。

即：

controlled solving

与

agentic solving.

请确认第一阶段是否接受固定 solver。

我的建议：**固定。**

否则很难干净地检验你的核心假设。

computational complexity 如何进入最终框架

你现在还没有完全决定 runtime / computational cost 是否属于 operational complexity。

我建议先不强行二选一，而采用**正交双向量**：

$$\mathbf{C}_{\text{OP}} = \begin{bmatrix} \mathbf{C}_{\text{model}} \\ \mathbf{C}_{\text{solve}} \end{bmatrix}.$$

其中：

$$\mathbf{C}_{\text{model}} = (C_{\text{semantic}}, C_{\text{coupling}}, C_{\text{ambiguity}}, C_{\text{feasibility}}, \dots)$$

而：

$$\mathbf{C}_{\text{solve}} = (n, m, \text{integrality}, \text{nonlinearity}, \text{treewidth}, \text{search-space proxy}, \text{runtime}, \text{gap}, \dots).$$

注意 runtime 本身可能不应该作为 complexity 的**定义量**，因为：

$$T = T(I, \text{solver}, \text{hardware}, \text{algorithm})$$

明显不是 model-independent。

更合理的是：

$$\text{problem structural complexity} \rightarrow \text{solver runtime}$$

就像：

$$C_{\text{op}} \rightarrow \text{LLM accuracy}.$$

换言之：

runtime 与 LLM accuracy 都作为 criterion variables / validation outcomes，而不是直接塞进 intrinsic complexity definition。

这样最符合你“类比 computational complexity”的原始目标。

请确认是否接受这一处理方式：computational difficulty 保留为独立轴，但 runtime 不直接进入 intrinsic definition。

我强烈推荐这个选择。

feasibility difficulty 具体指什么

你提出的这一点非常重要：

feasibility

似乎经常比 optimality 更容易暴露 LLM 的 formulation failure。

但 feasibility 有至少两种完全不同的意义。

第一种是**语义 feasibility burden**：

为了完整定义 feasible solution，需要同时正确表达多少规则、逻辑条件、边界条件和交叉依赖？

这是 modeling complexity。

第二种是**几何/数值 feasibility difficulty**：

例如 feasible region 极窄：

$$\frac{\text{Vol}(\mathcal{F})}{\text{Vol}(\mathcal{X})} \ll 1,$$

或约束 slack 很小、可行解稀疏、MIP feasible assignments 极少。

这是 mathematical/computational structure。

我建议明确拆成：

$$C_{\text{feas}}^{\text{semantic}}$$

和

$$C_{\text{feas}}^{\text{geometric}}.$$

前者描述“把所有 feasibility conditions 写对有多难”；后者描述“在正确模型下找到 feasible solution 有多难”。

这正好可以检验你的 Q9 假设：

$$C_{\text{feas}}^{\text{semantic}} \gg C_{\text{feas}}^{\text{geometric}}$$

是否是当前 solver-integrated agents 的主要 failure source。

请确认是否接受这一拆分。

modeling ambiguity 是否必须是 intrinsic ambiguity

你要求 model-independent，因此我不会把下面这种指标当成核心定义：

$$H(p_{\text{LLM}}(\text{formulation} \mid I)),$$

因为它显然依赖具体 LLM。

我准备把 ambiguity 定义为类似：

$$C_{\text{amb}} = \text{number/diversity of semantically plausible formal interpretations.}$$

例如：

“Each worker should work approximately five shifts.”

究竟是：

$$x_i = 5,$$

还是

$$4 \leq x_i \leq 6,$$

还是 soft constraint：

$$\min |x_i - 5|?$$

这种 ambiguity 是 instance 本身具有的。

实验上可以用多名 OR 专家独立 formalize，计算 inter-annotator divergence：

$$D_{\text{form}}$$

来验证。

请确认你是否接受“专家 formalization disagreement”作为 ambiguity 的 gold validation 方法之一。

benchmark 范围还有一个重要边界

你之前说希望覆盖：

“目前 LLM+OR benchmark 里实际出现的 problem classes。”

这个范围现在至少有两种可能解释。

较窄版本：automated optimization modeling benchmarks

包括：

- NL4Opt;
- OptiBench;
- IndustryOR / ORLM;
- OPT-Engine;
- Large-Scale-OR;
- Air-NRM;
- OR-LLM-Bench;
- ORAgentBench;
- OR-R1 / OR-LLM-Agent 对应 evaluation sets;
- 类似 end-to-end optimization agent benchmark。

较宽版本：还包括 algorithm-design / combinatorial-optimization research agents

例如 2026 年的 CO-Bench 更关注 agent 对 combinatorial optimization **算法设计与改进**的能力，而不是单纯把业务语义转成标准 solver formulation。 ³

如果纳入后一类，你的核心假设很可能天然被削弱，因为：

gold formulation $\nRightarrow$ standard solver call solves everything.

这反而可能成为很好的**边界条件**：

operational modeling complexity 适用于“solver-addressable optimization modeling tasks”，而 algorithm-discovery complexity 是另一种问题。

我的建议是：

主论文以 automated optimization modeling / solver-integrated OR benchmarks 为核心；CO algorithm-design benchmarks 作为外部边界分析，而不混入主 complexity definition。

请确认这一范围。

完整问题类型清单的排序原则

你问得很对：LP/MILP 不是和 scheduling/routing/facility location 同一级别的 taxonomy。

前者更多是数学规划形式：

$$LP, MILP, NLP, CP, SP, \dots$$

后者是应用/结构 problem class：

$$VRP, JSSP, FLP, inventory, \dots$$

因此报告中我准备给你做一个二维 taxonomy，而不是一个混杂清单：

$$\text{Problem structure} \times \text{mathematical formulation family}.$$

例如：

Problem family	常见 formulation
Transportation / flow	LP / network flow
Assignment / matching	LP / MILP
Facility location	MILP
Knapsack / allocation	IP / MILP
Scheduling	MILP / CP / CP-SAT
Routing	MILP / CP / specialized algorithms
Packing / cutting	MILP / column generation
Network design	MILP
Inventory	LP / DP / stochastic programming
Revenue management	LP / stochastic / dynamic optimization
Stochastic planning	two-stage / multistage SP
Robust optimization	RO / conic / MILP counterpart
Nonlinear engineering models	NLP / MINLP

之后再标记：

哪些已经出现在当前 LLM+OR benchmarks 中。

这种组织方式能避免把“LP”和“VRP”放在同一个 classification level。

这一点如果你没有异议，我会直接采用。

我准备使用的核心论文命题

如果上面的理解都成立，我会把最终研究工作的主命题组织为：

Operational Problem Complexity is the intrinsic burden required to transform

an operational specification into a complete, logically consistent, solver-addressable representation

更形式化地，对 operational instance：

$$I = (L, D, R),$$

其中：

- L ：自然语言任务描述；
- D ：数据；
- R ：business rules；

定义一个**不依赖具体 LLM**的向量：

$$\mathbf{C}_{\text{OP}}(I) = [C_{\text{ent}}, C_{\text{obl}}, C_{\text{local}}, C_{\text{global}}, C_{\text{cycle}}, C_{\text{hub}}, C_{\text{amb}}, C_{\text{feas}}, C_{\text{scale}}, \dots].$$

它应满足至少三个 empirical desiderata：

$$\frac{\partial P(\text{correct formalization})}{\partial C_j} < 0,$$

并且这种关系跨模型仍存在：

$$C_j \rightarrow \text{performance degradation} \quad \forall \text{ LLM families},$$

同时，在提供 gold formalization 后：

$$\left| \frac{\partial P(\text{success} \mid M^*)}{\partial C_{\text{model}}} \right| \ll \left| \frac{\partial P(\text{success} \mid I)}{\partial C_{\text{model}}} \right|.$$

最后再检验 solving complexity 是否仍产生独立作用：

$$P(\text{success}) = f(\mathbf{C}_{\text{model}}, \mathbf{C}_{\text{solve}}, \mathbf{C}_{\text{model}} \times \mathbf{C}_{\text{solve}}).$$

如果 $\mathbf{C}_{\text{solve}}$ 在 gold-model intervention 后几乎不影响成功率，你的强假设得到支持；如果它只在 routing / scheduling / large combinatorial instances 上出现显著效应，则可以得到一个更有价值的 **conditional theory**，而不需要强行证明“computational complexity 完全不重要”。

需要你确认的最终十项

为了避免你逐段回复，我把真正需要决定的地方压缩成十项。你可以直接回答类似：

1接受；2接受；3接受；4接受；5接受；6接受；7接受；8接受；9接受；10按你的建议。

项目	我准备采用的选择
定义层级	区分 Operational Problem Complexity 与 Formulation Complexity
语义结构	用 expert-defined canonical semantic entities + Semantic Obligation Hypergraph
自动化要求	第一篇论文允许 gold semantic annotation，不要求 complexity extraction 一开始就全自动
coupling	同时研究 semantic coupling 和 formulation/constraint coupling
gold-model实验	做 NL → semantic spec → gold math model → executable model 四级 intervention
solver控制	核心因果实验固定 solver / modeling framework；agentic solver selection 另做扩展
computational cost	solving complexity 独立保留；runtime 是验证 outcome，不直接进入 intrinsic definition
feasibility	区分 semantic feasibility burden 与 geometric/computational feasibility difficulty
ambiguity	intrinsic ambiguity 为主，用专家之间 formalization disagreement 做外部验证
benchmark范围	主体限定 solver-integrated automated optimization modeling；CO algorithm-design benchmark 作为边界分析

这十项一旦确定，后面的 literature review、数学指标、完整数值例子、benchmark taxonomy、gold-model intervention experiment、constraint-coupling synthetic experiment 和论文框架就可以建立在一个统一定义上，而不会在中途出现“我们到底在度量问题、formulation，还是某个 LLM 的能力”这种根本性混淆。

1 OPT-Engine: Benchmarking the Limits of LLMs in Optimization Modeling via Complexity Scaling

https://arxiv.org/abs/2601.19924?utm_source=chatgpt.com

2 Benchmarking the Limits of LLMs in Optimization Modeling ...

https://arxiv.org/html/2601.19924v2?utm_source=chatgpt.com

3 CO-Bench: Benchmarking Language Model Agents in ...

https://ojs.aaai.org/index.php/AAAI/article/view/40596/44557?utm_source=chatgpt.com